PYTHON FOR DATA ANALYSIS

A Practical, Hands-On Course Plan

Jupyter Notebooks • NumPy • pandas • Exploratory Data Analysis

Prerequisite	Python fundamentals already known — variables, loops, functions, data structures
Tools Used	Python 3, Jupyter Notebooks, NumPy, pandas, VS Code (optional)
Learning Style	Hands-on practice, real datasets, mini-projects after each module
Goal	Be able to load, clean, explore, and analyze real-world data with Python
Prepared by	**Abraham Chileshe**
Student	**Daniel**

Course Overview

This course picks up where Python fundamentals end and takes you deep into professional data analysis. You will write real code from day one, work with real messy datasets, and by the end you will have the skills in data analyst and data science roles.

The course is broken into 4 modules.

AT A GLANCE — All 4 Modules

MODULE 01

Jupyter Notebooks & Your Workflow

Set up your working environment, understand Jupyter Notebooks inside out, and develop good coding habits that will save you hours later.

MODULE 02

NumPy — Numerical Computing

Master arrays, math operations, and numerical data processing with NumPy, the backbone of all scientific computing in Python.

MODULE 03

pandas — Data Manipulation

The most important module. pandas is what you will use every single day as a data analyst. Learn to load, clean, filter, group, merge, and transform data.

MODULE 04

Exploratory Data Analysis (EDA)

Bring everything together. Learn how to explore unknown datasets, spot patterns, handle outliers, and produce insights from real-world data.

MODULE 01 — Jupyter Notebooks & Your Workflow

What This Module Is About

Jupyter Notebooks are the standard working environment for data analysis in Python. Every data analyst and data scientist uses them. This module teaches you to use them fluently and sets you up with good habits before we dive into the heavy libraries.

Topic	What You Will Learn
What Jupyter Notebooks are and why data people use them	Notebooks mix code, output, text, and charts in one document. Perfect for exploration and sharing results
Installing Jupyter and launching it	Install via pip or Anaconda, launch from terminal, and open your first .ipynb file
Understanding cells — code cells vs markdown cells	Code cells run Python. Markdown cells hold formatted text, headings, and notes. Learn to use both
Running cells, kernel basics, and restarting	Run cells with Shift+Enter, understand what the kernel is, and know when and how to restart it
Keyboard shortcuts that save you hours	Tab for autocomplete, Shift+Enter to run, Esc+B to add cell, Ctrl+/ to comment — essential muscle memory
Structuring a notebook for readable analysis	How to lay out a notebook so it tells a clear story: imports, data loading, exploration, cleaning, analysis, conclusions
Markdown formatting — headers, bold, lists, tables	Write clean documentation inside your notebook so your work is understandable to anyone
Magic commands (%timeit, %matplotlib inline, %%time)	Special Jupyter commands that let you time your code, display charts inline, and run shell commands
Exporting notebooks to HTML and PDF	Share your work as a polished report without anyone needing to run Python to read it
Common notebook pitfalls and how to avoid them	Why cell execution order matters, what hidden state means, and why you should Restart & Run All before sharing

MODULE 02 — NumPy: Numerical Computing

What This Module Is About

NumPy is the foundation of all numerical computing in Python. pandas itself is built on top of NumPy. You need to understand it to understand what is happening under the hood when you analyze data. This module takes you from zero to confident with arrays, math, and data manipulation at speed.

Topic 1: Introduction to NumPy

Topic	What You Will Learn
What NumPy is and why it is faster than plain Python	NumPy uses optimized C code under the hood. Operating on 1 million numbers takes milliseconds, not minutes
Importing NumPy — import numpy as np	The standard convention. Every data notebook starts with this line
NumPy vs Python lists — key differences	NumPy arrays are typed, fixed-size, and support vectorized operations. Lists are flexible but slow for math
Where NumPy fits in the Python data ecosystem	NumPy sits below pandas, scikit-learn, TensorFlow, and matplotlib — it powers all of them

Topic 2: Creating Arrays

Topic	What You Will Learn
np.array() — creating from a Python list	Convert any Python list into a NumPy array. This is how you bring data into NumPy
np.zeros(), np.ones(), np.full()	Create arrays pre-filled with zeros, ones, or any value — great for initializing results matrices
np.arange() — like range() but for arrays	Create evenly spaced integer arrays. np.arange(0, 100, 5) gives you 0, 5, 10, 15... up to 95

np.linspace() — evenly spaced float values	Create exactly N points between a start and end value. Essential for plotting and interpolation
np.random — random arrays for testing	np.random.rand(), np.random.randn(), np.random.randint() — create test data instantly
np.eye() and np.identity() — identity matrices	Create square matrices with 1s on the diagonal — used in linear algebra and machine learning
Creating 2D and 3D arrays	Arrays are not limited to one dimension. Understand how shape, axes, and dimensions work in NumPy

Topic 3: Array Attributes and Inspection

Topic	What You Will Learn
shape — dimensions of the array	arr.shape returns a tuple like (100, 5) meaning 100 rows and 5 columns. You will use this constantly
dtype — data type of elements	Know whether your array holds integers, floats, or strings. Wrong dtypes cause silent calculation errors
ndim — number of dimensions	1D, 2D, or 3D? ndim tells you how deep the array goes
size — total number of elements	Total count of values in the array regardless of shape
Changing dtype with astype()	Convert an array of strings to floats, or floats to integers — essential for data type corrections

Topic 4: Indexing and Slicing

Topic	What You Will Learn
1D indexing — positive and negative indices	Access elements by position. Negative indices count from the end: arr[-1] is the last element

2D indexing — rows and columns	arr[2, 4] gives row 2, column 4. Understanding this is essential before working with DataFrames
Slicing — extracting subarrays	arr[1:5, 0:3] extracts rows 1–4, columns 0–2. The same syntax powers pandas slicing
Boolean (fancy) indexing	Filter an array by condition: arr[arr > 50] returns only values above 50. This is vectorized filtering
np.where() — conditional element selection	Like an if-else applied to every element at once. Incredibly fast and clean for data transformations
Fancy indexing with integer arrays	Pass an array of indices to select specific rows or columns — useful for sampling and reordering

Topic 5: Array Operations and Math

Topic	What You Will Learn
Vectorized arithmetic — no loops needed	Add, subtract, multiply, divide entire arrays element by element without writing a single for loop
Broadcasting — operating on different shapes	NumPy automatically stretches smaller arrays to match larger ones. Powerful but confusing without explanation
Universal functions (ufuncs) — np.sqrt, np.exp, np.log, np.abs	Math functions that apply to every element instantly: square root, exponential, logarithm, absolute value
np.sum(), np.mean(), np.median(), np.std(), np.var()	Statistical aggregations — the bread and butter of data summarization
np.min(), np.max(), np.argmin(), np.argmax()	Find the smallest and largest values, and the positions (indices) where they occur
np.cumsum() and np.cumprod() — running totals	Calculate running totals along an array — great for time series and sales data

Axis parameter — row-wise vs column-wise operations	axis=0 operates down rows (per column), axis=1 operates across columns (per row). Critical concept

Topic 6: Reshaping and Manipulating Arrays

Topic	What You Will Learn
reshape() — changing array shape	Reshape a (100,) array to (10, 10) or (20, 5) — as long as the total number of elements stays the same
flatten() and ravel() — collapsing to 1D	Turn any multi-dimensional array into a simple 1D array. flatten() returns a copy, ravel() a view
transpose() and .T — flipping rows and columns	Swap the axes of an array. Essential for matrix math and reshaping data for models
np.concatenate(), np.vstack(), np.hstack()	Combine arrays vertically (add rows) or horizontally (add columns) — the array equivalent of database joins
np.split() — dividing arrays into pieces	Split an array into equal or custom-sized chunks — useful for batching data
np.sort() and np.argsort()	Sort arrays, and argsort() returns the indices that would sort the array — great for ranking

Topic 7: Linear Algebra with NumPy

You don't need to be a math expert here — but understanding these basics gives you a huge advantage when working with data later, especially in machine learning.

Topic	What You Will Learn
np.dot() and @ operator — matrix multiplication	Multiply two matrices together. The @ operator is the clean, modern way to write matrix multiplication
np.linalg.inv() — matrix inverse	Find the inverse of a square matrix — used in solving systems of equations and regression

np.linalg.det() — determinant	Check if a matrix is invertible. A determinant of zero means it is not — important in linear modeling
np.linalg.eig() — eigenvalues and eigenvectors	The foundation of PCA (dimensionality reduction). You don't need to understand the full math, just know it exists
np.linalg.solve() — solving linear systems	Solve Ax = b — a system of linear equations. Faster and more stable than computing the inverse

Topic 8: Handling Missing and Special Values

Topic	What You Will Learn
np.nan — Not a Number	np.nan is NumPy's way of representing a missing numeric value. Critical to understand before working with real data
np.isinf() and np.isnan()	Detect infinite or missing values in your array so you can handle them before analysis
np.nanmean(), np.nansum(), np.nanstd()	Versions of aggregation functions that simply ignore NaN values instead of returning NaN themselves
Replacing NaN values with np.where() or np.nan_to_num()	Substitute missing values with zeros, means, or other fill values as part of your cleaning pipeline

Topic 9: Saving and Loading NumPy Arrays

Topic	What You Will Learn
np.save() and np.load() — binary format (.npy)	Save your processed array to disk in NumPy's fast binary format and reload it instantly
np.savetxt() and np.loadtxt() — text/CSV format	Save and load arrays as plain text or CSV files — good for sharing with non-Python tools
np.savez() — saving multiple arrays at once	Bundle several arrays into one .npz file — like a zip archive for NumPy data

Module 3 — Project

Load a CSV of daily temperature readings using np.loadtxt(). Calculate mean, min, max, and standard deviation for each month. Identify days that were more than 2 standard deviations above average (heatwave detection). Reshape and transpose the data for a different reporting format. Save the cleaned result.

MODULE 03 — pandas: Professional Data Manipulation

What This Module Is About

This is the most important module in the entire course. pandas is what data analysts use every single day. It lets you work with real-world tabular data — the kind that lives in spreadsheets, databases, and CSV files — with power and speed. We will go deep.

Topic 1: Introduction to pandas

Topic	What You Will Learn
What pandas is and where it fits in data work	pandas is Python's answer to Excel — but programmable, reproducible, and scalable to millions of rows
Installing and importing pandas — import pandas as pd	The standard import. Confirmed convention across all data teams worldwide
The two core pandas structures: Series and DataFrame	A Series is one column. A DataFrame is a full table. Everything in pandas revolves around these two objects
How pandas is built on top of NumPy	Every column in a DataFrame is a NumPy array underneath. Understanding NumPy makes pandas faster to learn

Topic 2: pandas Series

Topic	What You Will Learn
Creating a Series from a list, dict, or NumPy array	A Series is a labeled 1D array. Create one from scratch or from existing data
Series index — the label system	Every value in a Series has a label. Labels can be numbers, strings, or dates — and you can use them to access data
Accessing values — positional (.iloc) vs label (.loc)	iloc uses position (0, 1, 2). loc uses the label. Knowing the difference prevents hours of debugging
Vectorized operations on a Series	Apply math, string operations, or conditions to an entire Series at once — no loops

Series methods — head, tail, describe, value_counts, sort_values	Quick inspection and summarization tools — you will use these in the first 30 seconds of looking at any dataset
Boolean filtering on a Series	series[series > 100] returns only values above 100. The pandas way to filter data
Missing values in Series — NaN handling	Detect with isna(), fill with fillna(), drop with dropna(). Real data is always missing some values

Topic 3: DataFrames — The Heart of pandas

Topic	What You Will Learn
Creating a DataFrame from a dictionary	Build a DataFrame from scratch by passing a dictionary of column name: list pairs
Reading data from files — read_csv(), read_excel(), read_json()	Load real-world data from CSV, Excel, and JSON files in one line. The most important pandas function
Writing data to files — to_csv(), to_excel()	Save your cleaned or transformed data back to a file for sharing or further processing
Inspecting a DataFrame — head(), tail(), info(), describe(), shape	The five commands you run on every new dataset. Understand size, column types, and basic statistics instantly
Accessing columns — df['column'] and df.column	Two ways to select a column. Always use bracket notation for safety with column names that have spaces or special characters
Accessing rows — .loc[] and .iloc[]	loc uses labels, iloc uses integer positions. These are the two most important accessors in all of pandas
The DataFrame index — setting, resetting, using as a label	The index labels your rows. Setting a meaningful index (like a date or ID) enables powerful lookups and time series operations

Topic 4: Selecting and Filtering Data

This is where you start doing real analysis — pulling exactly the rows and columns you care about.

Topic	What You Will Learn
Selecting single and multiple columns	df[['name', 'age', 'salary']] returns a DataFrame with only those three columns — your first step in focusing on relevant data

Boolean filtering — selecting rows by condition	df[df['age'] > 30] returns only rows where age is above 30. This is the most fundamental operation in data analysis
Filtering with multiple conditions using & and |	Combine conditions: df[(df['age'] > 25) & (df['city'] == 'Nairobi')]. Must use parentheses around each condition
The .query() method — readable filtering	df.query('age > 25 and city == "Nairobi"') — cleaner, more readable way to filter with string expressions
isin() — filtering by a list of values	df[df['country'].isin(['Kenya', 'Uganda', 'Tanzania'])] — filter for any value in a list
between() — filtering within a range	df[df['salary'].between(50000, 100000)] — clean range filter without combining two conditions manually
Filtering with str.contains() for text matching	Find rows where a text column contains a specific word or pattern — essential for messy text columns
Selecting a random sample — .sample()	Grab a random subset of rows for quick inspection or model testing

Topic 5: Cleaning Data — The Real Work

Experts say 80% of data work is cleaning. This is not an exaggeration. Real data is messy, inconsistent, and full of errors. This topic gives you the full toolkit.

Topic	What You Will Learn
Detecting missing values — isna(), isnull(), notna()	Find where your data has gaps. isna() returns a True/False DataFrame showing every missing cell
Counting missing values per column	df.isna().sum() tells you exactly how many values are missing in each column — your cleaning checklist
Dropping rows or columns with missing values — dropna()	Remove rows or columns that have too many missing values. Control the threshold with thresh parameter
Filling missing values — fillna()	Replace NaN with a fixed value, a column mean, a forward fill, or a backward fill — choose the right strategy for your data
Duplicate rows — duplicated() and drop_duplicates()	Detect and remove exact or partial duplicate records — a common issue in data collected from multiple sources

Fixing data types — astype(), pd.to_datetime(), pd.to_numeric()	Convert columns to the right type. Dates stored as strings, numbers stored as text — fix them all
Stripping whitespace and fixing text with .str methods	Leading/trailing spaces, inconsistent capitalization, and mixed cases — .str.strip(), .str.lower(), .str.title() fix them all
Replacing values — replace() and map()	Standardize inconsistent categories: replace 'M' with 'Male', 'F' with 'Female' across an entire column in one line
Renaming columns — rename() and direct assignment	Give columns clear, consistent names that follow your team's conventions or make code more readable
Handling outliers — identifying with IQR and z-score	Find extreme values that could skew your analysis and decide whether to cap, remove, or flag them
String cleaning with regex in pandas	Use regular expressions inside .str.replace() and .str.extract() for complex text cleaning tasks

Topic 6: Transforming Data

Once data is clean, you shape it into the form your analysis needs.

Topic	What You Will Learn
Creating new columns from existing ones	df['profit'] = df['revenue'] - df['cost'] — create derived columns with simple arithmetic or functions
apply() — applying any function to rows or columns	Apply a custom function to every row or every column. The most flexible transformation tool in pandas
map() — element-wise transformation on a Series	Apply a function or dictionary mapping to every single value in a column — clean and fast
applymap() / map() on DataFrames (element-wise)	Apply a function to every single cell in the entire DataFrame — useful for formatting or encoding
assign() — adding multiple columns at once	df.assign(tax=df.price*0.16, total=df.price*1.16) — add several new columns in a clean pipeline style

Cutting and binning with pd.cut() and pd.qcut()	Divide continuous numbers into categories: 0–18 becomes 'Child', 18–65 becomes 'Adult', etc.
Sorting with sort_values() and sort_index()	Sort your DataFrame by one or multiple columns, ascending or descending
Changing column order and dropping columns	Reorder columns to match a reporting template, or drop columns you no longer need
Setting and using a MultiIndex	Multiple layers of row labels — powerful for hierarchical data like country > city > neighborhood

Topic 7: Grouping and Aggregation

This is where pandas really earns its keep. Instead of summarizing an entire dataset, you split it into groups and summarize each group separately — sales by region, customers by age group, revenue by product.

Topic	What You Will Learn
groupby() — the foundation of grouped analysis	df.groupby('region')['sales'].sum() — split your data by a category and calculate a metric for each group
Multiple aggregation functions with agg()	Calculate mean, sum, count, min, and max all at once: groupby().agg(['mean', 'sum', 'count'])
Named aggregations — clean output column names	Use .agg(total_sales=('sales', 'sum')) to give your aggregated columns meaningful names
groupby on multiple columns	Group by more than one category: df.groupby(['year', 'region'])['revenue'].sum() — detailed breakdowns
transform() — broadcast group statistics back	Calculate a group mean and add it as a new column alongside each individual row — great for normalization
filter() — keep only groups matching a condition	Remove entire groups from your analysis: df.groupby('product').filter(lambda x: x['sales'].sum() > 10000)
pivot_table() — Excel-style pivot tables	Create cross-tabulation summaries with rows, columns, values, and aggregation functions — like a pivot table but in code
pd.crosstab() — frequency tables	Quick frequency counts between two categorical columns — great for understanding relationships in categories

resample() — grouping by time period	Group time series data by day, week, month, or year. Requires a DateTime index

Topic 8: Merging and Joining DataFrames

In real data work, your data never lives in one file. You will always need to combine data from multiple sources — like joining a customer table to a transactions table.

Topic	What You Will Learn
pd.concat() — stacking DataFrames	Stack DataFrames vertically (add rows) or horizontally (add columns) — like gluing tables together
pd.merge() — SQL-style joins	Combine two DataFrames on a shared column, just like a database JOIN
Inner join — only matching rows from both	Returns rows that appear in BOTH tables — the strictest join, no unmatched rows
Left join — all rows from left, matching from right	Keep all rows from the main table and fill NaN where the right table has no match
Right join — all rows from right, matching from left	The reverse of left join — less commonly used but important to understand
Outer join — all rows from both	Keep every row from both tables, filling NaN wherever there is no match — useful for auditing completeness
Joining on multiple columns	Sometimes two columns together identify a unique record — merge on both at once
Detecting and handling duplicates after merging	Merges can create unexpected duplicates. Always check the shape before and after a merge
df.join() — joining on index	Faster join method when using the DataFrame index as the key — good for time series data

Topic 9: Reshaping Data

Sometimes data comes in the wrong shape for analysis. Wide data needs to be tall (or the reverse). These tools fix that.

Topic	What You Will Learn
melt() — wide to long format (unpivoting)	Turn multiple value columns into a single column with a label. Converts wide spreadsheet data into tidy format

pivot() — long to wide format	Turn a long table with repeated categories into a wide table with one column per category
stack() and unstack() — rotating MultiIndex levels	Move DataFrame column labels to row index (stack) or row index to column labels (unstack)
pd.get_dummies() — one-hot encoding	Convert a categorical column into multiple binary (0/1) columns — required step before machine learning models
Tidy data principles — why shape matters for analysis	Understand the concept of tidy data (one observation per row, one variable per column) and why it makes analysis easier

Topic 10: Time Series with pandas

Dates and times are everywhere in data — sales records, sensor readings, financial data. pandas has excellent built-in tools for working with them.

Topic	What You Will Learn
pd.to_datetime() — converting strings to dates	Parse dates from strings in any format: '2024-01-15', '15 Jan 2024', '01/15/2024' all work
DateTime properties — .dt.year, .dt.month, .dt.day, .dt.weekday	Extract parts of a date: get the month of every transaction, or the day of week of every event
Setting a DatetimeIndex for time series operations	Once your date column is the index, grouping by month, resampling, and rolling averages become trivial
resample() — aggregating by time period	df.resample('M').sum() gives you monthly totals. 'W' for weekly, 'Q' for quarterly, 'A' for annual
rolling() — moving averages and smoothing	Calculate a 7-day rolling average to smooth out daily fluctuations and reveal trends
shift() — comparing to previous periods	df['prev_month'] = df['sales'].shift(1) — align previous values with current for period-over-period comparisons
Handling time zones with tz_localize() and tz_convert()	Essential when your data comes from multiple countries or systems with different time zones

Topic 11: Performance and Best Practices

Topic	What You Will Learn

Avoiding loops — vectorize everything possible	A loop over 1 million rows in Python takes minutes. The same operation as a vectorized pandas call takes milliseconds
Memory usage and reducing it with category dtype	Converting a string column with few unique values to 'category' dtype can cut memory usage by 95%
Using .query() and eval() for performance	For large DataFrames, .query() and pd.eval() are faster than standard boolean indexing
Chaining methods for readable pipelines	df.dropna().rename().query().groupby().agg() — chain operations instead of creating unnecessary intermediate variables
copy() vs view — avoiding SettingWithCopyWarning	Understand when pandas returns a copy of the data vs a view, and how to avoid the dreaded SettingWithCopyWarning

Module 4 — Project

You will receive three messy CSV files: a customer table, a sales transactions table, and a product catalog. Your job: merge them into a single clean dataset, handle all missing values and type errors, add derived columns (customer lifetime value, product category, transaction month), build a summary table of monthly revenue by product category, and export a clean Excel report.

MODULE 04 — Exploratory Data Analysis (EDA)

What This Module Is About

EDA is the process of getting to know a dataset you have never seen before. You are not yet trying to answer a specific question — you are learning what the data contains, what problems it has, and what questions it can answer. This is what a data analyst does in the first hours with any new dataset.

Topic 1: The EDA Process and Mindset

Topic	What You Will Learn
What EDA is and why it comes before analysis	Never skip EDA. Rushing to answers on dirty or misunderstood data leads to wrong conclusions with high confidence
Framing the right questions before exploring	Start with business questions: Who are our best customers? When do sales drop? What drives churn? Questions guide exploration
The iterative EDA cycle: look, question, verify, repeat	EDA is not a linear process. Every finding raises new questions. Embrace the cycle
Documenting findings as you go in Jupyter	Write markdown notes as you discover things. A notebook with no commentary is just code — not analysis

Topic 2: Understanding Your Dataset

Topic	What You Will Learn
Shape, columns, dtypes — the first 60 seconds	shape, info(), dtypes, columns — run these immediately on any new dataset
Counting unique values per column — nunique()	Tells you whether a column is an ID, a category, or a continuous number — critical for planning your analysis
value_counts() — frequency distribution of categories	See the most and least common values in a categorical column. Immediately reveals data quality issues

describe() for numeric and categorical columns	Get count, mean, std, min, max, and quartiles for all numeric columns at once. Your statistical snapshot
Identifying column roles — identifier, categorical, numerical, date	Classify every column before analyzing it. This determines which analysis technique applies

Topic 3: Univariate Analysis

Univariate means one variable at a time. Before looking at relationships, understand each column individually.

Topic	What You Will Learn
Distribution of numerical columns — histograms and density plots	Visualize how a number is spread across its range. Is it bell-shaped, skewed, bimodal? This matters for analysis
Central tendency — mean, median, mode and when to use each	Mean is sensitive to outliers. Median is robust. Know which one to report for which type of data
Spread — range, IQR, standard deviation, variance	How spread out is the data? Wide spread = high variability. Tight spread = consistency
Skewness and kurtosis — shape of the distribution	Skewness tells you if the tail is longer on one side. Kurtosis tells you how peaked or flat the distribution is
Outlier detection with IQR method and z-scores	Values more than 1.5 * IQR outside the quartile range are outliers by convention. z-score > 3 is another standard
Frequency tables for categorical columns	Count how many rows fall into each category. Unbalanced categories are a major issue in machine learning data

Topic 4: Bivariate Analysis

Bivariate means two variables at a time. This is where you start discovering relationships — which things move together, which things are independent.

Topic	What You Will Learn

Correlation — measuring linear relationships	df.corr() gives you a correlation matrix. Values close to 1 or -1 indicate strong relationships
Scatter plots — visualizing two numerical variables	Plot two numeric columns against each other to see if there is a pattern, trend, or cluster
Heatmaps — correlation matrices made visual	A color-coded grid of correlations. At a glance you can see which columns are strongly related
Box plots — numerical vs categorical	Compare the distribution of a number across different categories. 'How does salary differ by department?'
Bar charts — comparing category counts or means	The most common chart in business reporting. Totals, averages, or counts grouped by category
Pivot table EDA — cross-tabulating two variables	Quick two-dimensional summaries. 'Show me average sales by region and quarter in one table'

Topic 5: Multivariate Analysis

Topic	What You Will Learn
Grouped analysis — comparing distributions across segments	Group data by a category and compare key metrics. How does the distribution of revenue differ by country?
Pair plots — all pairwise scatter plots at once	A grid of scatter plots for every combination of numeric columns. Great for spotting structure in data quickly
Color and size encoding in scatter plots	Add a third variable as color and a fourth as size. See complex relationships in a single chart
Faceting — small multiples	Create the same chart for each category side by side. Compare patterns across segments at a glance

Topic 6: Data Quality Report

Topic	What You Will Learn
Systematically auditing completeness — % missing per column	Calculate and rank columns by missing rate. Decide which to impute, which to drop, which to investigate
Checking for duplicates — exact and near-duplicate records	Exact duplicates are easy. Near-duplicates (same person, slightly different name) require fuzzy matching
Detecting inconsistencies — contradictory or impossible values	Age of -5, salary of $0, event date before birth date — systematic checks catch these logical errors
Creating a data quality scorecard in pandas	A summary DataFrame showing completeness, uniqueness, and type correctness for every column in one place

Topic 7: Framing Analysis Problems

This topic is about thinking, not coding. It is the difference between a person who uses data tools and a person who solves problems with data.

Topic	What You Will Learn
Translating a business question into a data question	'Why are sales down?' becomes: which customer segment, product, or region changed most? Then you measure it
Identifying the right metric for the right question	Not every question needs an average. Sometimes you need a median, a ratio, a rate of change, or a percentile
Defining success criteria before analyzing	Decide before you run the analysis: what result would surprise you? What would confirm your hypothesis? This prevents motivated reasoning
Writing an analysis plan before writing code	A short bullet list of: dataset, question, cleaning steps, analysis approach, expected output. Saves hours of wandering
Communicating findings clearly in notebook markdown	Structure your notebook like a report: problem, data overview, key findings, caveats, recommendations

Topic 8: Real-World Dataset Challenges

Topic	What You Will Learn
Working with CSV files that have encoding issues	Files from different countries often have non-UTF-8 characters. Learn to detect and fix encoding problems
Inconsistent date formats across a dataset	One column with '01/15/2024', '2024-01-15', and 'January 15th' — standardize them all with pd.to_datetime
Mixed types in a column — numbers stored as strings	A salary column where most values are numbers but some are 'N/A' or 'unknown' — detect and clean mixed-type columns
Inconsistent category names across data sources	'New York', 'new york', 'NY', 'N.Y.' are all the same city. Fuzzy matching and standardization strategies
Working with very large files that don't fit in memory	chunksize parameter, reading in batches, filtering on load — strategies for datasets too large for RAM
Real EDA walkthrough on a public dataset	A full EDA from scratch on a real dataset: Kaggle housing prices, NYC taxi data, or WHO health statistics

Module 5 — Capstone Project

Choose a real dataset from Kaggle or a public API. Write a full exploratory data analysis notebook from scratch. Your notebook must include: dataset overview and initial inspection, data cleaning documentation (what was wrong, how you fixed it), univariate analysis for at least 5 key columns, bivariate analysis discovering at least 3 interesting relationships, a data quality report, and a summary of your 5 most important findings written in plain English. This is the project you will show employers.